Summary Report: AoI–PLP Analysis in IIoT Networks

In this project, I looked at how different network factors affect the trade-off between Age of Information (AoI) and Packet Loss Probability (PLP) in an IIoT system. The main goal was to understand how things like transmission probability, number of nodes, channel quality, and capture threshold impact both how fresh the data is and how reliable it is.

From the beginning, it was pretty clear that these variables don't really work on their own. They all kind of interact with each other. For example, if you increase transmission probability, updates happen more often so AoI improves, but at the same time, more devices are trying to send data, which causes collisions and increases PLP. Also, when you have more nodes in the network, everything gets more crowded, which makes both AoI and PLP worse. Channel quality also matters because if the signal is bad, a lot of transmissions fail anyway.

Because of this, just using the original features wasn't really enough. So I added some feature engineering, especially things like **network load**, which combines transmission probability and number of nodes. This helped represent how busy the network actually is. I also added some interaction and non-linear features since the system clearly isn't linear. After doing this and removing some extreme outliers, the model started to perform a lot better.

I first used a Random Forest model, which got an R^2 of about 0.27 for AoI. It wasn't amazing, but it was definitely better than before and showed that the model was starting to pick up some real patterns. It also helped show that features like network load were actually important.

Then for the bonus part, I built a simple deep learning model that predicts both AoI and PLP at the same time. This model did better overall. The AoI prediction improved to an R^2 of 0.34, and the PLP prediction was actually really good with an R^2 of 0.78. I think this worked better because AoI and PLP are related, so predicting them together makes more sense.

From all of this, it seems like the main idea is that you need to balance the network. You don't want transmission probability too high or too low, and you need to manage how many devices are active at once. Things like reducing congestion and improving channel quality can help a lot. These ideas would be useful in real systems like smart factories or healthcare monitoring, where you need data to be both fast and reliable.

Overall, this project showed me that just throwing data into a model isn't enough. Once I actually started thinking about how the system works and added better features, the results improved a lot. And the deep learning model showed that more advanced methods can do even better when the relationships are more complex.